

Ticketing System - Technical Lead Take-Home

Scope & intent: This test focuses on **architecture and scaffolding**, **NOT** a finished app. Please show how you'd structure and set up the project so others can build on it. Well-documented **stubs** (see [What is a Stub?](#)) and clear decisions are sufficient.

Submission Instructions

- Fork the [starter repository](#) to your own GitHub account.
- Complete the assignment in your fork. You are encouraged to delete, reorganize, or replace any files, directories, or structure as you see fit.
- When you're finished, **send us the link to your GitHub repository**.
- Make sure all code, scripts, and documentation are committed to your repo—**no external links, gists, or attachments**.

Project Scenario

You are the first engineer on a new ticketing system for live events (concerts, conferences, etc.). The product will have a web client, and the expectation is that the codebase will be expanded by future team members. You are responsible for laying the groundwork so the project is easy to scale, maintain, and onboard new engineers.

Starter Repository

We have provided a [starter repository](#) with an example directory structure and placeholder files. **This scaffold is only an example and exists to clarify expectations around deliverables. You are encouraged to delete, reorganize, or replace any files, directories, or structure as you see fit.** Use your preferred stack, tools, and frameworks.

- Please pay attention to the **README.md** files in each directory for additional notes and guidance.
- You may rip out or completely change any aspect of the starter repo.

What You Need to Deliver

1. Backend

- Expose a small API for a ticketing domain with **stubs** for:
 - Authentication
 - Event listing
 - Ticket purchase
 - User management: read, create, delete and update

- Maintain clear separation between **request handling**, **domain/business logic**, and **data access** (even if some layers are stubbed).
- Include an **authentication and authorization layer** (middleware or equivalent) that illustrates token/session verification and a basic role check (e.g., `user` vs `admin`). Stubbing `req.user` or equivalent for development is fine.
- **Replicability and Portability:** The full backend (including all dependencies such as database, storage, and supporting services) must be easily runnable locally by a new developer on any OS, using a **single command** (or minimal set of commands) and **without requiring any global installations**. Use any packaging or orchestration you consider standard for scalable teams.
- **Provisioning & Infrastructure:** Provide scripts/configuration that automate provisioning of all core backend infrastructure (locally and/or in the cloud). Your provisioning approach should reflect:
 - **Scalability:** Resources should be provisioned in a way that supports scaling out (e.g., supports statelessness, horizontal scaling, load balancing, etc.) suitable for modern cloud environments.
 - **Security:** Demonstrate awareness of secure deployment practices for your chosen platform—e.g., network isolation, proper credential management, least-privilege access, and secure endpoints.
 - **Extensibility:** Structure the provisioning so that new services/resources can be added without major rework.
 - You do not need to implement a production-ready setup, but the configuration/scripts (e.g. infrastructure as code, CLI script, or cloud console template, etc.) should *illustrate* these concerns (comments/TODOs acceptable).
- At a minimum, include automation for provisioning:
 - **Compute runtime** (where your API runs)
 - **Managed database** (e.g., PostgreSQL)
 - **Object storage** (for assets)
 - **Authentication provider** (real or stubbed) Your automation should be **repeatable, documented, and suitable for scaling out**, with a clear flow for setup and (optionally) teardown.
- Provide a `.env.example` showing required environment variables (e.g., `DATABASE_URL`, `STORAGE_BUCKET`, `AUTH_ISSUER`, `AUTH_AUDIENCE`, etc.), with brief notes on secrets management.
- **Note:** You do **NOT** need to fully implement all features. Focus on structure, architecture, and rationale.

2. Frontend

- Create a minimal frontend scaffold (web, framework up to you). Organize code so that it's clear where routing, page-level logic, and shared services/utilities belong.
- Include **stubbed files/components/pages** for:
 - Login
 - Event listing (public)

- Ticket purchase (protected/gated)
- Focus on **structure and integration points**, *not* UI/UX polish.
- Demonstrate, with code or clear pseudocode, how the frontend would:
 - **Authenticate** users (e.g., login flow, token storage)
 - **Call backend APIs** for event/ticket data and ticket purchase. (It's fine if the backend is a stub or returns mock data.)
 - **Handle access control** (e.g., gating pages/routes by auth state, showing/hiding actions based on user role)
- The project should be **simple to build and serve locally**, with clear instructions and minimal setup (ideally a single command to run).
- Show or describe the **build → deployment flow** for the frontend:
 - You *don't* need to set up CI/CD, but you *should* provide either a script or a documented command that builds a production-ready artifact and (optionally) serves it locally or describes how you would deploy it (e.g., “run `npm run build` and then `npm run preview`” or “output in `dist/` can be hosted on any static file server/cloud storage”).
- **Note:** You do **NOT** need to implement all the functionality. Stubs, pseudocode, or comments outlining intended logic are sufficient, as long as your structure and approach are clear.

3. Portability and Onboarding

- Your entire solution—backend, database, and any supporting services—must be runnable locally by a new developer, regardless of their OS or what's installed on their machine.
- There should be a **single command** (or minimal set of commands) that brings up all required services for local development, with no manual setup of runtimes, databases, or infrastructure dependencies on the host system.
- All services and resources required by the app (e.g., API server, database, object storage, authentication provider/stub) should be included in this local setup, each either as a container, isolated service, or repeatable script.
- Your top-level `README.md` should provide step-by-step instructions for getting started locally, and, if possible, describe how the same setup could be provisioned in a cloud environment using your provided automation/configs.
- Make sure your approach supports a new developer onboarding with zero guesswork and minimal install steps.
- In your documentation, **justify** your stack and packaging choices, and call out any trade-offs (e.g., if you chose to mock object storage locally, or stub out auth in development).
- Bonus: If your approach allows for easy teardown/reset of the full environment, mention how this can be done.

4. Security & Extensibility

- Show awareness of security and best practices.
- Organize code and docs in a way that makes it easy for future developers to add features (e.g., new endpoints, payment processor, analytics).
- *You are not expected to implement everything, but you should show where and how you'd expand the system e.g. payment service, etc.*

5. Timebox

- **Spend no more than X hours total, over up to N days.**
- Focus on architecture, structure, and documentation—not feature-completeness.
- Clearly mark any features you leave as stubs or incomplete, and briefly describe how you would implement them if you had more time.

6. Deliverables Checklist

Your final submission should include the following:

- ☐ **Backend** A directory containing:
 - Project manifest (e.g., `package.json`, `go.mod`, etc.)
 - Clear code structure with API stubs for authentication, event listing, ticket purchase, and user management
 - Authentication middleware with role-based access stub (e.g., user vs admin)
 - A `.env.example` file with all required environment variables
- ☐ **Frontend** A directory containing:
 - Project manifest (e.g., `package.json`)
 - Stubbed pages or components for login, event listing, and ticket purchasing
 - Example service/API layer demonstrating how the frontend communicates with the backend (can be pseudocode)
 - Local development build/run instructions
- ☐ **Local Development Setup** Your full project should be easily runnable by another developer on any OS, without needing to install runtimes or dependencies globally. Use **any packaging or orchestration approach that ensures this out-of-the-box experience**, such as:
 - A one-liner to spin up the backend and database
 - An isolated environment for local development that mirrors production deployment While the exact tooling is up to you, we're looking for **portability, isolation, and ease-of-onboarding**—you're encouraged to apply whatever standard solution you would use in a real-world team environment.
- ☐ **Infrastructure Automation** A script/configuration that scalably and securely provisions:
 - A compute runtime (e.g., virtual machine, serverless function, or container runner)
 - A database (e.g., PostgreSQL)
 - Object storage (e.g., for tickets, receipts, images)

- An authentication provider (real or stubbed) You may use any method you're comfortable with—Terraform, shell scripts with CLI tools, etc.—as long as the process is documented and reproducible.

☐ Documentation

- A top-level `README.md` that:
 - Explains how to run and build the project locally
 - Describes your architecture and technology decisions
 - Highlights any areas left stubbed and what you would do with more time
- Any per-directory `README.md` files as appropriate for backend, frontend, infra, or scripts

Notes

- **You are free to throw away or reorganize any part of the provided scaffold.**
- **You do NOT need to implement a working production system—this is about design and scaffolding.**
- **Well-documented stubs/mockups are perfectly fine.**
- Focus on how you would set up the project so that *future developers* can build on it efficiently.

Definition: What is a Stub?

Stub: In this assignment, a “stub” is a file or function with the correct name and location in your project, but only partially implemented or filled with placeholder logic. It exists to show **where** code will go and **how** parts of the system connect, even if the full logic is not implemented yet.

Example (pseudocode/TypeScript):

TypeScript

```
// Example stub for a login endpoint
export function login(req, res) {
  // TODO: implement authentication
  return res.status(501).send({ message: "Login not implemented yet." });
}
```

Comments and TODOs inside stubs are encouraged to explain your intended behavior and next steps.